

Design of a Four-Bit Binary to Decimal Conversion

Module 1 Basics: Number Systems & Logic Gates — the Language of the Machine

A quick, foundational look at how a 4-bit binary value becomes a decimal number — using only what Module 1 covers: binary/hex, logic gates and simple Boolean design.



Team



P.V.Gowtham Sai

2620030105



A.Vishal Yadav

2620030106



Vaideesh

2620030110



N.Vishwak

2620030104



R.Sai Midhilesh

2620030084

What Is a 4-Bit Binary to Decimal Conversion?

- A computer stores every number as bits — 0s and 1s. A 4-bit number is simply 4 of these bits written together, like 1101.
- "Converting to decimal" means finding the ordinary base-10 number that binary value represents — so humans can read it easily.
- Our project builds a small circuit that does exactly this: it takes 4 input bits and lights up exactly one output line, showing which decimal value (0 – 15) they represent.



$$1101_2 = 13_{10}$$

What This Project Uses From Module 1

1 Number systems

Binary and hexadecimal, and how to read a 4-bit value.

2 Logic gates

AND, OR, NOT and friends — the building blocks of the circuit.

3 Truth tables & Boolean expressions

Turning "what the circuit should do" into gates.

4 Tooling

Logisim/Digital to build and simulate the circuit we design.

Number Systems: Binary, Decimal, Hex

Binary

1101

Base 2 — the only alphabet hardware actually stores.

Decimal

13

Base 10 — what humans read and write.

Hexadecimal

0xD

Base 16 — a short, convenient way to write binary.

A 4-bit value covers 16 combinations (0000 – 1111), matching decimal 0 – 15 — exactly the range our converter needs to handle.

Representing Our Values

- Our values are unsigned — decimal 0 to 15, so no negative numbers and no sign bit.
- Two's complement (the standard way to represent negative numbers) isn't needed for this project, since nothing here goes below zero.
- Binary addition and "overflow" also don't come up here — we're only decoding an existing 4-bit value, not adding numbers together.

0 — 15

the full range of an
unsigned 4-bit value

0000 → 0

1111 → 15

Logic Gates: The Basics

Every digital circuit, including ours, is built from a small handful of gates.

AND

Output is 1 only if both inputs are 1.

OR

Output is 1 if at least one input is 1.

NOT

Flips the input: 0 becomes 1, 1 becomes 0.

NAND

AND, then NOT — output is 0 only if both inputs are 1.

NOR

OR, then NOT — output is 1 only if both inputs are 0.

XOR / XNOR

1 only when inputs differ (XOR) or match (XNOR).

Our converter uses just NOT and AND — two of the simplest gates on this list.

Truth Table for Our Converter

All 16 combinations of the 4-bit input ABCD and their decimal output

ABCD	Y	ABCD	Y	ABCD	Y	ABCD	Y
0000	0	0100	4	1000	8	1100	12
0001	1	0101	5	1001	9	1101	13
0010	2	0110	6	1010	10	1110	14
0011	3	0111	7	1011	11	1111	15

From Truth Table to Circuit

- Pick the row you want: for decimal 5, the input is 0101 (A=0, B=1, C=0, D=1).
- Write it as a Boolean expression: only this exact combination should turn output Y5 on.
- Build it with gates: NOT flips the 0s, AND combines everything into one output.

$$Y5 = A' \cdot B \cdot C' \cdot D$$

Read as: NOT A, AND B, AND NOT C, AND D — true only for input 0101 (decimal 5).

NAND Universality

Any logic function — AND, OR, NOT, even our whole converter — can be built using only NAND gates.

- This is why NAND is called a "universal gate" — a real chip only needs one basic building block.
- Our design uses NOT and AND directly for clarity, but everything we've built could be re-expressed entirely in NAND gates.

Tooling & Workflow

1

Logisim / Digital

The visual logic simulator where we place gates, wire them up, and test every input combination.

2

Ripes

The RISC-V visualiser we'll return to later, once this logic becomes part of a larger machine.

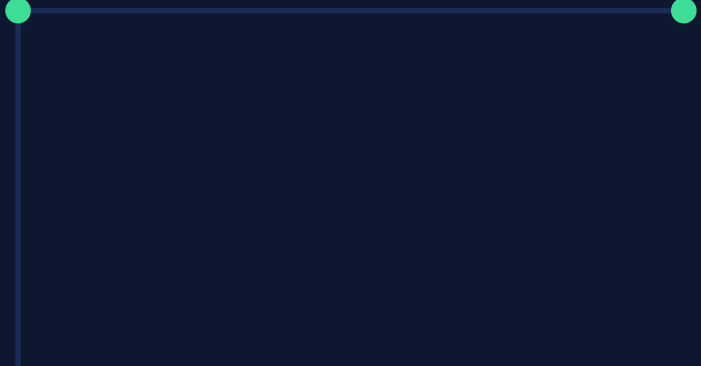
3

Git

A repository to track the circuit files and notes as the project grows across modules.

Module 1 Takeaways

- Binary, decimal and hex are just different ways to write the same number — our project works entirely in binary and decimal.
- A handful of logic gates (here, just NOT and AND) are enough to build a real circuit.
- A truth table turns directly into a Boolean expression, and then into gates — our core design move.
- Logisim/Digital, Ripes and Git are the tools and habits we'll keep using across every module.



Thank You

Design of a Four-Bit Binary to Decimal Conversion — Module I Basics